



8

Code Splitting Using Lazy Components and Suspense

Code splitting has been a significant part of React applications for many years, even before official support was included in the **React API**. The evolution of React has brought about APIs that are specifically designed to assist in code-splitting scenarios. Code splitting becomes crucial when dealing with large applications containing a vast amount of JavaScript code that needs to be delivered to a browser.

In the past, **monolithic JavaScript bundles** containing an entire application could cause usability issues due to long initial page load times. Thanks to code splitting, we now have much more granular control over how code is transferred from the server to the browser. This gives us ample opportunities to optimize load-time **User Experience (UX)**.

In this chapter, we will revisit how to implement this in your React applications by using the `lazy()` API and the `Suspense`

components. These features are very powerful tools in the React toolbox. By gaining a thorough understanding of how these components function, you'll be fully equipped to seamlessly integrate code splitting into your applications.

We'll cover the following topics in this chapter:

- Using the `lazy()` API
- Using the `Suspense` component
- Avoiding **lazy components**
- Exploring **lazy pages** and **routes**

Technical requirements

You can find the code files of this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter08>.

Using the lazy API

There are two pieces involved with using the `lazy()` API in React. First, there's bundling components into their own separate files so that they can be downloaded by the browser separately from other parts of the application. Second, once you have created the bundles, you can build React components that are lazy: they don't download anything until they are needed. Let's look at both of these.

Dynamic imports and bundles

The code examples in this book use the **Vite** tooling for creating bundles. The nice thing about this approach is that you don't have to maintain any bundle configuration. Instead, bundles are created for you automatically, based on how you import your modules. If you're using the plain `import` statement (not to be confused with the `import` method) everywhere, your app will be downloaded all at once in one bundle. When your app gets bigger, there will likely be features that some users never use or don't use as frequently as others. You can use the `import()` function to import modules on demand. By using this function, you're telling Vite to create a separate bundle for the code that you're importing dynamically.

Let's look at a simple component that we might want to bundle separately from the rest of the application:

```
export default function MyComponent() {  
  return <p>My Component</p>;  
}
```

Now, let's take a look at how we would import this module dynamically using the `import()` function, resulting in a separate bundle:

```
function App() {  
  const [MyComponent, setMyComponent] = React.useState<() => React.ReactNode>(  
    () => () => null  
  );  
  React.useEffect(() => {  
    import("./MyComponent").then((module) => {  
      setMyComponent(() => module.default);  
    });  
  }, []);  
  return <MyComponent />;  
}
```

When you run this example, you'll see the `<p>` text rendered right away. If you open the browser dev tools and look at the network requests, you'll notice that a separate call is made to fetch the bundle containing the `MyComponent` code. This happens because of the call to `import("./MyComponent")`. The `import()` function returns a promise that resolves with the module object. Since we need the default export to access `MyComponent`, we reference `module.default` when we call `setMyComponent()`.

The reason why we're setting a component as the `MyComponent` state is that when the `App` component renders for the first time, we don't have the `MyComponent` code loaded yet. Once it loads, `MyComponent` will reference the proper value, which results in the correct text being rendered.

Now that you have an idea of how bundles get created and are fetched by the app, it's time to see how the `lazy()` API greatly simplifies this process for us.

Making components lazy

Instead of manually handling the promise returned by `import()` by returning the default export and setting state, you can lean on the `lazy()` API. This function takes a function that returns an `import()` promise. The return value is a lazy component that you can just render. Let's modify the `App` component to use this API:

```
import * as React from "react";
const MyComponent = React.lazy(() => import("./MyComponent"));
function App() {
  return <MyComponent />;
}
```

The `MyComponent` value is created by calling `lazy()`, passing in the dynamic module import as an argument. Now, you have a separate bundle for your component and a lazy component that loads the bundle when it's first rendered.

In this section, you learned how code splitting works. You learned that the `import()` function handles bundle creation

for you. You also learned that the `lazy()` API makes your components lazy and handles all of the gritty work of importing components for you. But there's one last thing we need, the `Suspense` component, to help display placeholders while components are loading.

Using the Suspense component

In this section, we'll explore some of the more common usage scenarios of the `Suspense` component. We'll look at where to place `Suspense` components in your component tree, how to simulate latency when fetching bundles, and some of the options available to us to use as the fallback content.

Top-level Suspense components

Lazy components need to be rendered inside of a `Suspense` component. However, they do not have to be direct children of `Suspense` though, which is important because this means that you can have one `Suspense` component handle every lazy component in your app. Let's illustrate this concept with an example. Here's a component that we would like to bundle separately and use lazily:

```
export default function MyFeature() {  
  return <p>My Feature</p>;  
}
```

```
}
```

Next, let's make the `MyFeature` component lazy and render it inside of a `MyPage` component:

```
const MyFeature = React.lazy(() => import("./MyFeature"));  
function MyPage() {  
  return (  
    <>  
      <h1>My Page</h1>  
      <MyFeature />  
    </>  
  );  
}
```

Here, we're using the `lazy()` API to make the `MyFeature` component lazy. This means that when the `MyPage` component is rendered, the code bundle that contains `MyFeature` will be downloaded because `MyFeature` was also rendered. What's important to note with the `MyPage` component is that it is rendering a lazy component (`MyFeature`) but isn't rendering a `Suspense` component. This is because our hypothetical app has many page components, each with its own lazy components. Having each of these components render its own `Suspense` component would be redundant. Instead, we can render one `Suspense` component inside of our `App` component, like so:

```
function App() {  
  return (  
    <React.Suspense fallback={"loading..."}>  
      <MyPage />  
    </React.Suspense>  
  );  
}
```

While the `MyFeature` code bundle is being downloaded, `<MyPage>` is replaced with the fallback text passed to `Suspense`. So, even though `MyPage` isn't lazy itself, it renders a lazy component that `Suspense` knows about and will replace its children with the fallback content while this happens.

So far, we haven't really been able to see the fallback content that displays while our lazy components load their code bundles. This is because when developing locally, these bundles load almost instantly. To be able to see fallback component and loading process, you can enable throttling in **Network** tab of the dev tools:

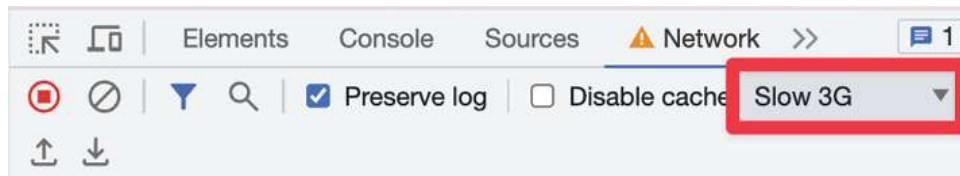


Figure 8.1: Enabling throttling in the browser

This setting emulates a slow internet connection. Instead of loading instantly, the page will be rendering for a few seconds and you will see a **loading ...** fallback.

In the next section, we'll look at an approach to use loading spinners as a fallback component.

Working with spinner fallbacks

The simplest fallback that you can use with the `Suspense` component is some text that indicates to the user that something is happening. The `fallback` property can be any valid React element, which means that we can enhance the fallback to be something more visually appealing. For example, the `react-spinners` package has a selection of `spinner` components, all of which can be used as a fallback with `Suspense`.

Let's modify the `App` component from the last section to include a `spinner` from the `react-spinners` package as the `Suspense` fallback:

```
import * as React from "react";
import { FadeLoader } from "react-spinners";
import MyPage from "../MyPage";
function App() {
```

```
return (  
  <React.Suspense fallback={<FadeLoader color="lightblue" />}>  
    <MyPage />  
  </React.Suspense>  
);  
}
```

The `FadeLoader` component will render a `spinner` that we've configured with a color value of `lightblue`. The rendered element of the `FadeLoader` component is passed to the `fallback` property. With the Slow 3G throttling, you should be able to see the `spinner` when you first load the app:



Figure 8.2: The image rendered by the loader component

Now, instead of text, we're showing an animated `spinner`. This likely provides a user experience that your users are more accustomed to. The `react-spinners` package has several `spinners` for you to choose from, each of which has several

configuration options. There are other spinner libraries that you can use or implement on your own.

In this section, you learned that you can use a single `Suspense` component that will display its fallback content for any lazy components that are lower in the tree. You learned how to simulate latency during local development so that you can experience what your users will experience with your `Suspense` fallback content. Finally, you learned how to use components from other libraries as the fallback content to provide something that looks better than plain text.

In the next section, you'll learn about why it doesn't make sense to make every component in your app a lazy component.

Avoiding lazy components

It might be tempting to make most of your React components lazy components that live in their own bundle. After all, there isn't much extra work that needs to happen to set up separate bundles and make lazy components. However, there are some downsides to this. If you have too many lazy components, your app will end up making several HTTP requests to fetch them: at the same time. There's no benefit to having separate bundles for components used on the same part of the app. You're better off trying to bundle components together in a way that one

HTTP request is made to load what is needed on the current page.

A helpful way to think of this is to associate **pages** with **bundles**. If you have lazy page components, everything on that page will also be lazy yet bundled together with other components on the page. Let's build an example that demonstrates how to organize our lazy components. Let's say that your app has a couple of pages and a few features on each page. We don't necessarily want to make these features lazy if they're all going to be needed when the page loads. Here's the `App` component that shows the user a selector to pick which page to load:

```
const First = React.lazy(() => import("./First"));
const Second = React.lazy(() => import("./Second"));
function ShowComponent({ name }: { name: string }) {
  switch (name) {
    case "first":
      return <First />;
    case "second":
      return <Second />;
    default:
      return null;
  }
}
```

The `First` and `Second` components are the pages that make up our app, so we want them to be lazy components that load their bundles on demand. The `ShowComponent` component renders the appropriate page when the user changes the selector:

```
function App() {  
  const [component, setComponent] = React.useState("");  
  return (  
    <>  
      <label>  
        Load Component:{" "}   
        <select  
          value={component}  
          onChange={(e) => setComponent(e.target.value)}  
        >  
          <option value="">None</option>  
          <option value="first">First</option>  
          <option value="second">Second</option>  
        </select>  
      </label>  
      <React.Suspense fallback={<p>loading...</p>}>  
        <ShowComponent name={component} />  
      </React.Suspense>  
    </>  
  );  
}
```

Next, let's look at the first page and see how it's composed, starting with the `First` component:

```
import One from "./One";
import Two from "./Two";
import Three from "./Three";
export default function First() {
  return (
    <>
      <One />
      <Two />
      <Three />
    </>
  );
}
```

The `First` component pulls in three components and renders them: `One`, `Two`, and `Three`. These three components will be part of the same bundle. While we could make them lazy, there would be no point, as all we would be doing is making three HTTP requests for bundles at the same time instead of one.

Now that you have a better understanding of how to map the page structures of your application to bundles, let's look at another use case where we use a router component to navigate around our app.

Exploring lazy pages and routes

In the *Avoiding lazy components* section, you saw where to avoid making components lazy when there is no benefit in doing so. The same pattern can be applied when you're using `react-router` as the mechanism to navigate around your application. Let's take a look at an example. Here are the imports we'll need:

```
const First = React.lazy(() => import("./First"));
const Second = React.lazy(() => import("./Second"));
function Layout() {
  return (
    <section>
      <nav>
        <span>
          <Link to="first">First</Link>
        </span>
        <span> | </span>
        <span>
          <Link to="second">Second</Link>
        </span>
      </nav>
      <section>
        <React.Suspense fallback={<FadeLoader color="lightblue" />}>
          <Outlet />
        </React.Suspense>
      </section>
    </section>
  );
}
```

```
);  
}  
export default function App() {  
  return (  
    <Router>  
      <Routes>  
        <Route path="/" element={<Layout />}>  
          <Route path="/first" element={<First />} />  
          <Route path="/second" element={<Second />} />  
        </Route>  
      </Routes>  
    </Router>  
  );  
}
```

In the preceding code, we have two lazy page components that will be bundled separately from the rest of the app. The fallback content in this example uses the same `FadeLoader` `spinner` component that was introduced in the *Working with spinner fallbacks* section.

Note that the `Suspense` component is placed beneath the navigation links. This means that the fallback content will be rendered in the spot where the page content will eventually show when it loads. The children of the `Suspense` component are the `Route` components that will render our lazy page components: for example, when the `/first` route is activated, the

`First` component is rendered for the first time, triggering the bundle download.

That brings us to the end of this chapter.

Summary

This chapter was all about code splitting and bundling, which are important concepts for large React applications. We started by looking at how code is split into bundles in your React applications by using the `import()` function. Then, we looked at the `lazy()` React API and how it helps to simplify loading bundles when components are rendered for the first time. Next, we looked more deeply at the `Suspense` component, which is used to manage content while component bundles are being fetched. The `fallback` property is how we specify the content to be shown while bundles are being loaded. You typically don't need more than one `Suspense` component in your app, as long as you follow a consistent pattern for bundling pages of your app.

In the next chapter, you'll learn how to use the **Next.js** framework to handle rendering React components on the server. The Next.js framework allows you to create pages that act as React components and can be rendered on the server and in the browser. This is an important capability for applications that

need good initial page load performance: that is, all applications.